Pinnacle Properties Group — Transaction Management Team: Technology Context

Services Owned by This Team

Service	Responsibility	Technology
Transaction Service	Transaction CRUD, milestone state machine, TC assignment, milestone advancement (with document checks)	Node.js 20 + PostgreSQL 15
Document Service	Document checklist management, DocuSign envelope creation and webhook processing, S3 storage	Node.js 20 + PostgreSQL 15 + S3
Commission Engine	All four plan-type calculations, cap accumulator events, CDA PDF generation	Node.js 20 + PostgreSQL 15
CDA Service	CDA PDF generation, DocuSign routing to Broker of Record, countersignature webhook processing	Node.js 20 + S3 + DocuSign
Disbursement Service	Stripe Connect payout initiation, instant vs standard payout logic, idempotency	Node.js 20 + Stripe API
QBO Sync Service	QuickBooks Online journal entry creation per disbursement; reconciliation status tracking	Node.js 20 + QBO API
MLS Sync Service	Polls NTREIS, HAR, Stellar MLS every 15 minutes; emits listing status events	Node.js 20 + RESO Web API (ECS cron job)

Frontends Owned by This Team

Client	Technology	Notes
TC Transaction Portal	React 18 SPA (TypeScript)	Milestone timeline, document checklist, DocuSign status tracking, dual agency workflow
Finance / Admin Console	React 18 SPA (role-gated views)	CDA queue, disbursement queue, QBO sync status, cap progress report
Broker of Record Console	React 18 SPA (role-gated)	CDA countersignature queue (DocuSign deep-link); dual agency approvals; audit trail

---

Key Technical Constraints

Transaction Milestone State Machine

- Transaction lifecycle is modeled as a formal state machine (using XState v5)
- State transitions are validated server-side by the Transaction Service; invalid transitions return HTTP 400

- Milestone advancement check: before allowing `advanceMilestone`, Transaction Service calls Document Service to verify all required documents for the current milestone have `status = approved`
- If any required document is missing or not approved, Transaction Service returns HTTP 409 with a list of missing/unresolved documents

Commission Engine Design

- **Idempotency**: commission calculation is a pure function — same `transactionId`, `agentId`, `gci`, `planVersion`, `ytdBrokeragePaid`, `anniversaryDate` inputs always produce identical output
- Every calculation is persisted with: `calculationId` (UUID), `transactionId`, `inputs` (JSON), `outputs` (JSON), `planVersion`, `inputHash` (SHA-256 of inputs), `calculatedAt`
- The Commission Engine **never mutates existing calculation records** — a new record is created for each recalculation; the latest calculation is the authoritative one
- Cap boundary calculations: if a transaction causes the YTD accumulator to cross the cap threshold mid-transaction, the engine splits the transaction into: (a) the portion subject to the brokerage split, and (b) the portion retained 100% by the agent
- **100% test coverage required** on all Commission Engine calculation paths (enforced in CI)

DocuSign Integration

- All DocuSign envelopes are created server-side (Document Service or CDA Service); no client-side DocuSign SDK
- **DocuSign Connect webhook**: all envelope status events (`completed`, `declined`, `voided`) are received via a webhook endpoint (HMAC signature verified before processing)
- Webhook idempotency: each event includes an `envelopeId`; Document Service checks for duplicate events before updating document status
- CDA countersignature: CDA Service listens for `cda_envelope.completed` event; triggers `cda.countersigned` EventBridge event consumed by Disbursement Service

Stripe Connect Payouts

- Disbursement Service initiates payouts using the agent's Stripe `accountId` (stored in Agent Service; fetched via internal API call before payout)
- Idempotency key = disbursement record ID (UUID); prevents duplicate payouts on Stripe retries
- **Standard ACH payout**: `method = standard` (2 business days); initiated via Stripe Connect `POST /v1/payouts`
- **Instant payout**: `method = instant` (same-day, 1.5% fee); only available if agent's Stripe account is instant-payout-enabled
- Payout failure (e.g., incorrect bank account): Disbursement Service emits a `disbursement.failed` event; Notification Service alerts the agent and Finance team

MLS Sync Service

- Runs as a scheduled ECS Fargate task every 15 minutes (EventBridge Scheduler)
- RESO Web API authentication: OAuth 2.0 client credentials per MLS (NTREIS, HAR, Stellar have separate credentials)
- For each active listing ID tracked in the Transaction Service, the sync job fetches the current `ListingStatus` from the MLS
- Status change detected → emits `mls.listing.status.changed` event to EventBridge → Transaction Service consumes and updates milestone if the new status maps to a different milestone stage

- Failure handling: if 3 consecutive sync cycles fail for a listing (tracked in DynamoDB per `listingId`), an alert is emitted to Notification Service; manual re-sync endpoint available in the Finance/Admin console

TRID Compliance Enforcement

- When a transaction reaches **Clear to Close** milestone, the Document Service checks:
  1. Is this a mortgaged purchase transaction? (transaction type = `PURCHASE` AND `hasMortgage = true`)
  2. Is a Closing Disclosure document present with a `deliveredAt` timestamp?
  3. Is the scheduled closing date ≥ 3 business days after `deliveredAt`? (business day calculation excludes weekends and US federal holidays via a holiday calendar lookup)
- If check (3) fails, the milestone advancement is blocked with a compliance error; the TC is shown the earliest permissible closing date

---

External Integrations (this team)

Integration	Protocol	Direction	Notes
DocuSign	REST + Connect webhook	Document/CDA Service ↔ DocuSign	Envelope creation, status webhooks; HMAC signature verified
Stripe Connect	REST + webhook	Disbursement Service ↔ Stripe	Payout initiation; `payout.paid`, `payout.failed` webhooks
QuickBooks Online	OAuth 2.0 + REST	QBO Sync Service → QBO	Journal entry per disbursement; refresh token stored in Secrets Manager
NTREIS / HAR / Stellar MLS	RESO Web API (OAuth 2.0)	MLS Sync Service → MLS	Listing status polling (read-only Phase 1); RESO write-back Phase 2
Agent Service	Internal REST	Disbursement Service → Agent Service	Fetch agent's Stripe accountId before payout; commission plan at contract date
S3 (SSE-KMS)	HTTPS (pre-signed URL)	Document Service → S3	Document upload and secure retrieval; 7-year lifecycle policy

---

Non-Functional Requirements (this team)

Category	Requirement
Milestone enforcement	Hard-block enforced server-side; missing document list returned in HTTP 409 response
Commission calculation	Idempotent; 100% unit test coverage; calculation result stored within 2 seconds of `transaction.closed` event
DocuSign webhook processing	Webhook acknowledged within 3 seconds; HMAC verification before any state change

Stripe payout idempotency	Idempotency key = disbursement UUID; no double payout on webhook retries
MLS sync	Every 15 minutes; status change reflected in DealDesk within 30 minutes of MLS update
TRID enforcement	Holiday calendar updated annually; enforcement check runs before every Clear-to-Close milestone advance
QBO sync	Disbursement journaled to QBO within 15 minutes of `disbursement.completed` event
Document retention	S3 Lifecycle Policy: Standard (years 1–3), Glacier (years 3–7); no manual deletion permitted